

THE UNIVERSITY OF ZAMBIA
SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

FINAL YEAR PROJECT REPORT

ACADEMIC YEAR 2026

**SVDS: A Camera-Based Stolen Vehicle Detection
System for the Zambia Police Service — The
Backend, Database and Security Component**

A Final Year Project Report Submitted to the
Department of Computing and Informatics
in partial fulfilment of the requirements for the award of

[DEGREE PROGRAMME — confirm]

Student Name: [STUDENT NAME — Role 2: Backend and Security]

Computer Number: [COMPUTER NUMBER]

Supervisor: [SUPERVISOR NAME]

Date of Submission: [DATE OF SUBMISSION]

DECLARATION

I declare that this final year project report is my own work and has been prepared in accordance with the academic requirements of the University of Zambia. All sources of information, ideas, data, software, images, and other material that are not my own have been appropriately acknowledged and referenced.

Where this project was undertaken as part of a group, I have clearly identified my individual contributions and have not presented the work of other group members as my own.

Student Name: [STUDENT NAME — Role 2: Backend and Security]

Computer Number: _____

Signature: _____

Date: _____

APPROVAL

This final year project report has been submitted for examination with the approval of the project supervisor.

Supervisor Name: _____

Signature: _____

Date: _____

Departmental/Examiner Approval (if applicable):

ABSTRACT

Stolen-vehicle recovery in Zambia depends largely on police officers chancing to recognise a plate number from a paper report or radio bulletin. This project, carried out by a group of four students, built SVDS, a Stolen Vehicle Detection System on top of the VRRS vehicle registry. The public files stolen-vehicle reports through a web application, and a camera node reads plates from a phone's video stream and checks them against active reports, so connected police officers are alerted in real time.

This individual report covers the backend and security component: the FastAPI service, its PostgreSQL data model, and its authentication and access control. The backend has four tables (users, reports, alerts, audit log), stores passwords as bcrypt hashes, issues signed JSON Web Tokens that carry the user's role, and enforces three roles (reportee, police, admin) on the server with reusable dependencies plus per-report ownership rules. It implements the report lifecycle (under review, missing, found), an audit trail, and the endpoint the camera node calls to ask whether a plate is stolen, which matches plates regardless of case and spacing.

The backend was verified with 31 automated tests, all of which pass, run against an in-memory SQLite database. Targeted probe tests then confirmed weaknesses that the tests did not cover: tokens remain valid after a user is deactivated, demoted or logged out; report status accepts arbitrary values; the public plate-check endpoint can be called repeatedly to create false alerts; and failed logins are not limited. These findings are documented with recommended fixes and have not been fixed in this project. The report concludes that the backend meets its access-control goals for tokens as issued but should not be used with real police data until the recommendations are applied.

Keywords: role-based access control; JSON Web Token; FastAPI; PostgreSQL; API security

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

*Appear addressed in the working tree: **F-1** and **F-2** (every request now re-reads the user from the database, so a deactivated or demoted user's old token is rejected, and the role comes from the database, not the token) and **F-9** (CORS_ORIGINS is now read). Still open as far as the changes show: F-3, F-4, F-5, F-6, F-7 (logout does not end a session by itself), F-8 and F-10.*

ACKNOWLEDGEMENTS

[Write acknowledgements here: supervisor, group members (AI/vision, real-time and integration, frontend), family, and anyone who helped collect plate images. The author must write this; it is personal.]

CONTENTS

Preliminary pages: Declaration · Approval · Abstract · Acknowledgements · List of Abbreviations and Acronyms · Note on Group Projects

1 Introduction — 1.1 Background and Context · 1.2 Problem Statement · 1.3 Project Aim · 1.4 Project Objectives · 1.5 Research Questions · 1.6 Scope of the Project (In Scope, Out of Scope) · 1.7 Significance · 1.8 Limitations · 1.9 Organisation of the Report

2 Literature Review — 2.1 Introduction · 2.2 Key Concepts and Theories · 2.3 Existing Systems / Related Work · 2.4 Comparative Analysis · 2.5 Research / Knowledge Gap · 2.6 Conceptual Framework · 2.7 Chapter Summary

3 Methodology — 3.1 Introduction · 3.2 Research / Development Approach · 3.3 Requirements / Data Collection · 3.4 System Architecture / Research Workflow · 3.5 Tools and Technologies · 3.6 Ethical and Legal Considerations · 3.7 Evaluation Method · 3.8 Chapter Summary

4 System Analysis and Design — 4.1 Introduction · 4.2 Stakeholder and User Analysis · 4.3 Functional Requirements · 4.4 Non-Functional Requirements · 4.5 Use Cases / User Stories · 4.6 System / Solution Architecture · 4.7 Database / Data Design · 4.8 Interface / Interaction Design · 4.9 Algorithms / Models / Technical Design · 4.10 Chapter Summary

5 Implementation / Development — 5.1 Introduction · 5.2 Development Environment · 5.3 System Components / Modules · 5.4 Key Implementation Details · 5.5 Integration · 5.6 Challenges and Solutions · 5.7 Individual Contribution · 5.8 Chapter Summary

6 Testing, Results and Evaluation — 6.1 Introduction · 6.2 Testing Strategy · 6.3 Test Cases and Results · 6.4 Experimental / Performance Results · 6.5 User Evaluation · 6.6 Analysis of Results · 6.7 Objective-by-Objective Evaluation · 6.8 Individual Results · 6.9 Chapter Summary

7 Discussion — 7.1 Introduction · 7.2 Interpretation of Findings · 7.3 Comparison with Existing Work · 7.4 Contributions of the Project · 7.5 Limitations and Implications · 7.6 Chapter Summary

8 Conclusion and Recommendations — 8.1 Introduction · 8.2 Summary of the Project · 8.3 Conclusions · 8.4 Achievement of Objectives · 8.5 Recommendations · 8.6 Future Work

References

Appendices — A Individual Contribution Statement · B User Manual / Installation Guide · C Additional System Designs / Diagrams · D Additional Test Cases and Results · E Data Collection Instruments · F Additional Code / Configuration · G Other Supporting Material

[Page numbers are not shown here. In Word, apply heading styles and use References → Table of Contents to generate a numbered contents page, and a List of Figures and List of Tables the same way.]

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
ALPR	Automatic Licence Plate Recognition
API	Application Programming Interface
CUDA	Compute Unified Device Architecture (NVIDIA GPU computing platform)
CORS	Cross-Origin Resource Sharing
ORM	Object-Relational Mapper
OWASP	Open Worldwide Application Security Project
RBAC	Role-Based Access Control
REST	Representational State Transfer
DBMS	Database Management System
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection over Union
JWT	JSON Web Token
mAP	mean Average Precision
MJPEG	Motion JPEG (video stream format)
ML	Machine Learning
OCR	Optical Character Recognition
SVDS	Stolen Vehicle Detection System
UI	User Interface
VRRS	Vehicle Registry and Reporting System
YOLO	You Only Look Once (object detector family)

NOTE ON GROUP PROJECTS

Students who worked on a project as a group are strongly encouraged to prepare their own individual final reports rather than submitting identical reports. The project, system, dataset, or overall research problem may be shared, but each student's report should clearly demonstrate that student's own understanding, responsibilities, work, results, and contribution to the project.

In particular, group-project students should:

- state the overall group project and explain how the individual work fits within it;
- clearly identify their own responsibilities and deliverables;
- focus detailed analysis, design, implementation, experiments, testing, and/or evaluation on their own contribution;
- acknowledge the work completed by other group members where it is discussed;
- include an *Individual Contribution Statement* in the appendices; and
- not claim another group member's work, results, code, data collection, or design decisions as their own.

Chapter 1: Introduction

1.1 Background and Context

Vehicle theft is a persistent problem for law enforcement in Zambia. Recovering a stolen vehicle depends heavily on chance: an officer at a checkpoint or on patrol happens to recognise a plate number from a paper report or a radio bulletin. Vehicle records, incident reports and eyewitness descriptions are typically kept on paper or in disconnected station-level spreadsheets, so a report filed at one police station is invisible to an officer elsewhere.

This report describes **SVDS**, the Stolen Vehicle Detection System, built by a group of four students on top of the **VRRS** (Vehicle Registry and Reporting System). Members of the public file stolen-vehicle reports through a web application. A camera node reads number plates from a phone's video stream and asks the server whether each plate belongs to an active report. Police officers are alerted when it does.

Every part of that flow depends on one server-side component: the **backend**. It stores the reports and accounts, decides who may see and change what, and answers the camera's question "is this plate stolen?". This report is the individual report of the student responsible for that component and for **security and access control**.

1.2 Problem Statement

Stolen-vehicle information in Zambia is fragmented and slow to reach the people who could act on it. A system that lets the public report a theft and lets cameras check plates against those reports needs a central service that:

1. keeps one authoritative record of reports, users and detections;
2. is trustworthy about *who* is asking, so that public users, officers and administrators each see only what their role permits; and
3. keeps a record of who did what, so that decisions about a report can be traced.

Without these, a reporting system either leaks sensitive vehicle and personal data, or lets the wrong person activate, alter or delete a report. The specific problem addressed here is **designing and building a database-backed API that stores reports securely, enforces role-based access on the server, and answers plate-match queries from the camera node**.

1.3 Project Aim

Group aim. To design, implement and evaluate SVDS: a vehicle registry and reporting web application integrated with a real-time, camera-based licence plate detection pipeline, for the Zambia Police Service and the public it serves.

Individual aim (Backend and security). To build and evaluate the backend API, its PostgreSQL data model, and its authentication and access-control layer.

1.4 Project Objectives

Group objectives

1. Role-based accounts (Reportee, Police, Admin) with scoped access.
2. Public filing and tracking of stolen-vehicle reports (`under_review` → `missing` → `found`).
3. A live view of alerts, the registry and analytics for police.
4. Administrator control of accounts, system health and the audit trail.
5. Detection and reading of plates from a live camera feed, matched against active reports.
6. Real-time alerts to connected police clients.

Individual objectives (Backend and security)

No.	Objective	Measure of success
B1	Design and implement the relational data model (users, reports, alerts, audit log)	Four tables with keys and constraints created from the ORM models
B2	Implement registration, login and password storage without plaintext passwords	Passwords stored as bcrypt hashes; login issues a signed, expiring JWT
B3	Enforce three-role access control on the server for every route	A test matrix shows each role reaching only its permitted routes
B4	Implement the report lifecycle with validation and ownership rules	Reportees see only their own reports; only police can activate or resolve; only admins can delete
B5	Implement the plate-match endpoint used by the camera node	Tolerates case and spacing differences; matches only active reports; records an alert
B6	Write an audit record for state-changing actions	Each such action produces a row naming the actor and target
B7	Verify the above with automated tests and a security review	All backend tests pass; weaknesses found are recorded honestly

1.5 Research Questions

1. Can role-based access control be enforced consistently on the server, using a signed token that carries the user's role, so that no route depends on the front end to hide it?
2. What security weaknesses remain in such a design once it works, and which of them can be demonstrated rather than merely suspected?

1.6 Scope of the Project

1.6.1 In Scope

- **This report:** the FastAPI application structure, the SQLAlchemy models and PostgreSQL schema, the schemas that validate input and output, registration and login, password hashing, JWT creation and checking, the role-checking dependencies, user management, the report lifecycle endpoints, the check-plate endpoint, audit logging, and the tests of these.
- **Group:** the whole system, including the components in Chapter 5's table that belong to other members.

1.6.2 Out of Scope

- The detection model and camera node (AI/vision member).
- The WebSocket broadcast, camera status and live-feed proxy, and system health (real-time and integration member). The check-plate endpoint *calls* the broadcast but does not implement it.
- The React portals and analytics charts (frontend member).
- Production hosting, TLS termination, backups and database migrations.

1.7 Significance of the Project

The backend is what makes SVDS safe to use. The Zambia Police Service gains a single record of reports that officers in any station can search, with each action traceable to an account. The public gain a way to file a report and follow its status without being able to see anyone else's. The project also documents, honestly, where a small role-based API of this kind is weak, which is useful to anyone extending it.

1.8 Limitations

- Security testing was limited to automated tests and targeted probes run against an in-memory test database. No penetration test, load test or external review was carried out.

- The tests run on SQLite, not on the PostgreSQL database used in practice. One PostgreSQL-specific function is emulated in the test set-up (§6.2).
- No formal stakeholder interviews are recorded, so requirements are the group's own analysis.
- Several weaknesses found (§6.4) were **documented, not fixed**, within the project's time.

1.9 Organisation of the Report

Chapter 2 reviews the concepts (REST, JWT, bcrypt, RBAC) and related practice. Chapter 3 describes the method, tools, ethics and evaluation. Chapter 4 gives the analysis and design of the backend. Chapter 5 describes the implementation and the student's contribution. Chapter 6 presents the tests, the security probes and an objective-by-objective evaluation. Chapter 7 discusses the findings. Chapter 8 concludes. The appendices contain the Individual Contribution Statement, installation guide, extra diagrams, test output and code.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the concepts and technologies behind the SVDS backend: REST APIs, token-based authentication, password hashing, role-based access control, relational data modelling and audit logging. It then looks at existing systems and identifies the gap. Section numbers follow the template.

2.2 Key Concepts and Theories

2.2.1 REST APIs and FastAPI

A REST API exposes resources (here: reports, users, alerts) at URLs and uses HTTP methods (GET, POST, PATCH, DELETE) to act on them. **FastAPI** is a Python framework that builds such APIs from type-annotated functions. It validates request bodies with **Pydantic** models and generates interactive documentation (Swagger UI at `/docs`). Its **dependency injection** mechanism lets a route declare what it needs (a database session, the current user), which is how access control is attached to routes in this project.

2.2.2 Relational databases and ORMs

A relational database stores data in tables linked by keys, and constraints (uniqueness, foreign keys, ON DELETE CASCADE) keep the data consistent. **PostgreSQL** is an open-source relational database. **SQLAlchemy** is an object-relational mapper (ORM): tables are described as Python classes and queries are written in Python. A weakness of `create_all()` schema creation, relevant here, is that it creates missing tables but never alters existing ones.

2.2.3 Password hashing

Passwords must never be stored in plaintext. A password *hash function* turns a password into a fixed string that cannot practically be reversed. General-purpose hashes are too fast; password hashing needs a deliberately slow, salted algorithm. **bcrypt** is such an algorithm: each hash includes a random salt and a cost factor, so identical passwords give different hashes and brute-force guessing is slow. The `passlib` library provides a `CryptContext` wrapper.

2.2.4 JSON Web Tokens

A **JSON Web Token (JWT, RFC 7519)** is a signed string carrying *claims*, such as the user id (sub), the role, and an expiry (exp). The server signs it with a secret key (here HMAC-SHA256, HS256) and can later verify that it was issued by the server and has not been altered, without a database lookup. This makes the server *stateless* about sessions. The trade-off is that a token stays valid until it expires, even if the user is later deactivated or demoted, unless the server adds extra checks.

2.2.5 Role-based access control

In **role-based access control (RBAC)**, permissions attach to roles and users are given roles. SVDS uses three, in a hierarchy: **reportee** (public) $\subset$ **police** $\subset$ **admin**. The principle of *least privilege* says each role should have only what it needs. Checks must run on the server: hiding a button in the user interface is not access control, because a client can send any request.

2.2.6 Audit logging

An **audit log** records who performed which significant action on what, and when. It supports accountability and later investigation, which matters in a police system where a report can be activated, altered or deleted.

2.3 Existing Systems / Related Work

- **Django REST Framework / Flask** are common Python alternatives. They offer built-in or add-on authentication and permission classes. FastAPI was chosen here for its automatic validation and documentation and for explicit dependency-based access checks.
- **OAuth 2.0 / OpenID Connect providers** (for example Keycloak or Auth0) provide hardened login, refresh tokens, revocation and multi-factor authentication. They are the mature answer for production but add a service to run and configure; this project implements a small self-contained equivalent.
- **The OWASP guidance** (the OWASP Top 10 and the API Security Top 10) lists the risks a system like this should be checked against: broken access control, cryptographic failures, identification and authentication failures, security misconfiguration, and excessive data exposure. The security review in Chapter 6 uses these categories as a checklist.
- **Police records systems** in the public domain are mostly closed. Published descriptions concentrate on data content, not on how access is enforced, so no detailed comparison is possible.

Note on sources. This chapter relies on the projects' public documentation and standards, listed in the References. No performance or security figures from other systems are quoted.

2.4 Comparative Analysis

Criterion	This project (FastAPI + own JWT)	Django REST Framework	Hosted identity provider
Setup effort	Low	Medium	Medium to high
Token revocation / refresh	Not implemented	Available via add-ons	Built in
Multi-factor authentication	No	Add-on	Built in
Understandable end to end by a student team	Yes	Yes	Less (external service)
Suitable for production as is	No (see Chapter 6)	With configuration	Yes

The table compares design properties, not measured performance.

2.5 Research / Knowledge Gap

Tutorials show how to add JWT login to FastAPI, but rarely show a *verified* account of what such a design does and does not protect against. The gap this report addresses is a working, tested role-based backend for a stolen-vehicle registry, together with **demonstrated** (not assumed) limitations of the token design.

2.6 Conceptual Framework

The backend follows a layered flow in which each request passes the same gates:

```
HTTP request → CORS check → JWT decoded (get_current_user)
                → role check (require_role) → input validation (Pydantic)
                → business rule (ownership, status) → database → audit record →
response
```

Authentication answers "who is this?", authorisation answers "may they do this?", and the audit record answers "what happened?".

2.7 Chapter Summary

FastAPI, SQLAlchemy/PostgreSQL, bcrypt and JWT are standard, well-documented building blocks. A JWT-based RBAC design is simple and stateless but leaves token-lifetime weaknesses unless extra checks are added. The backend uses this design and the following chapters evaluate it, including where it falls short.

Chapter 3: Methodology

3.1 Introduction

This chapter explains how the backend was developed and evaluated: the approach, how requirements were obtained, the architecture, tools, ethical and legal considerations, and the evaluation method.

3.2 Research / Development Approach

The project was built **iteratively**. The web application (backend and frontend) and the camera-node pipeline were two work-streams that met at one point, POST /alerts/check-plate. For the backend, work proceeded in slices: data model first, then authentication, then role checks, then the report lifecycle, then the plate-match endpoint and the alert and analytics routes. Each slice was tried through FastAPI's interactive /docs page and later covered by automated tests.

A **design-and-verify** approach was used for security: after the features worked, the code was read for weaknesses, each suspected weakness was turned into a short probe test, and only behaviour actually observed was recorded as a finding (§6.4).

```
Requirements → Design → Implementation → Integration → Testing / Security review  
→ (issues found? back)
```

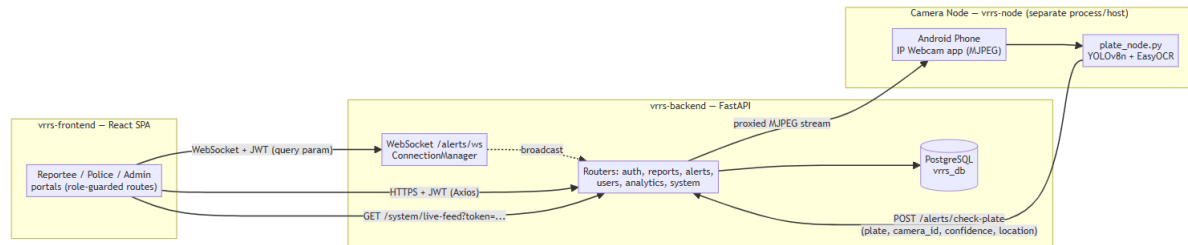
3.3 Requirements / Data Collection

Requirements came from (a) the gap in Chapter 1, (b) the natural three-way split of people who touch a stolen-vehicle report (the person who filed it, the officer investigating it, the administrator), and (c) the needs of the other components: the camera node needs one endpoint, the front end needs role-scoped listings, and the real-time member needs a hook to broadcast alerts. No stakeholder interviews or surveys are recorded, so the requirements are the group's own analysis.

The data handled by the backend is personal and sensitive: names, telephone numbers, national ID numbers, police badge numbers, vehicle details and locations. Test data used in this report is synthetic (for example jane@test.local); no real personal data appears in the tests.

3.4 System Architecture / Research Workflow

SVDS has three independently runnable processes that communicate only over the network: vrrs-backend (this report), vrrs-frontend and vrrs-node.



3.5 Tools and Technologies

Area	Technology
Language	Python
Web framework	FastAPI 0.111.0 with Uvicorn 0.29.0
Database	PostgreSQL, accessed through psycopg2-binary 2.9.9
ORM	SQLAlchemy 2.0.30 (Alembic 1.13.1 is listed in requirements.txt but not wired up)
Validation	Pydantic 2.7.1
Password hashing	passlib[bcrypt] 1.7.4 with bcrypt 4.1.3
Tokens	python-jose[cryptography] 3.3.0, algorithm HS256
Configuration	python-dotenv (.env)
Testing	pytest with FastAPI TestClient and in-memory SQLite
OS	Windows 11

Versions are those pinned in vrrs-backend/requirements.txt.

3.6 Ethical and Legal Considerations

- **Personal data.** The system stores personal and police data. Access is limited by role, passwords are hashed, and every state-changing action is audited. A deployment would also need a legal basis and a retention policy under Zambian data protection law; those are not addressed by the code.

- **Data exposure.** Reportees can read only their own reports. Police can read all reports and all users. The response models limit which user fields are returned (§4.4); the password hash is not among them.
- **Accountability.** The audit log gives a trace of who activated, resolved or deleted a report and who changed a role, but it has gaps (§6.4).
- **False matches.** A wrong plate match could point officers at an innocent driver. The backend matches only against reports police have activated, and officers can flag alerts as false positives.
- **Unauthenticated endpoint.** `check-plate` is public because the camera has no user identity. This is a deliberate trade-off with a demonstrated consequence (§6.4).
- **Secrets.** The signing key and database password live in a `.env` file that is not committed; `.env.example` documents the keys with placeholder values.
- **Responsible testing.** Security probes were run only against the project's own code using an in-memory test database, never against a live deployment or anyone else's system.

3.7 Evaluation Method

1. **Automated tests** (pytest, 31 tests) covering authentication, role access, reports, the plate-match endpoint and analytics.
2. **Targeted security probes:** short, temporary tests written to confirm or refute suspected weaknesses. They were run, their output recorded, and then removed so that the test suite stays a suite of intended behaviour.
3. **Code review** against the OWASP categories listed in §2.3.
4. **Objective-by-objective review** against B1 to B7 in §1.4.

No load or performance testing and no user evaluation were carried out.

3.8 Chapter Summary

The backend was built iteratively in slices and verified by automated tests plus probe tests that check suspected weaknesses. Data handled is sensitive; test data is synthetic.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the analysis and design of the backend. Stakeholders and the architecture are shared by the group; the requirements, data design, API design and security design are those of the **backend and security component**, the subject of this report.

4.2 Stakeholder and User Analysis

Stakeholder	Interest	Needs from the backend
Reportee (public)	Report a theft, follow its status	Register and log in; file and view only their own reports; edit own profile
Police officer	Investigate, activate and resolve reports, act on alerts	See all reports; activate and mark found; view and flag alerts; view analytics
Administrator	Manage accounts and oversee the system	Create police accounts, change roles, deactivate or delete users, delete reports, read the audit log
Camera node	Ask whether a plate is stolen	One public endpoint that answers CLEAR or STOLEN
Front-end and real-time developers	Build on the API	A stable, documented API; a broadcast hook for alerts

4.3 Functional Requirements

ID	Requirement
FR-B1	Public registration creates reportee accounts only; police and admin accounts are created by an administrator.
FR-B2	Login with email and password returns a signed token, the role and a redirect path; deactivated accounts cannot log in.
FR-B3	Passwords are stored only as bcrypt hashes; a password must be at least 6 characters.
FR-B4	Every protected route requires a valid token; role rules are checked on the server.
FR-B5	A reportee can file a report; a new report starts as <code>under_review</code> . An active (missing) report for the same plate blocks a new one.

ID	Requirement
FR-B6	A reportee can list and read only their own reports; police and admin can list and read all, with search and status filters and paging.
FR-B7	Police can activate a report (<code>under_review</code> → <code>missing</code>) and mark it found (recording who and when). Police can edit report details.
FR-B8	Only an administrator can delete a report, and deleting it also deletes its alerts.
FR-B9	An administrator can create police accounts, change a user's role, deactivate or reactivate a user, and delete a user. An administrator cannot change, deactivate or delete their own account.
FR-B10	<code>POST /alerts/check-plate</code> normalises the plate, matches only <code>missing</code> reports, records an alert and triggers the real-time broadcast on a match.
FR-B11	Police can list alerts, mark them read, and flag false positives.
FR-B12	State-changing actions write an audit record naming the actor, action and target.
FR-B13	Users can view and edit their own profile and change their password (the current password must be supplied).

4.4 Non-Functional Requirements

- **Security:** no plaintext passwords; signed, expiring tokens; server-side role checks; responses limited to the fields a client needs (`UserOut` excludes the password hash, national ID and badge number).
- **Validation:** telephone numbers must match `+260 XX XXXXXXX`, national IDs `NNNNNN/NN/N`, enforced by Pydantic patterns at registration.
- **Consistency:** database constraints back the business rules (unique email, foreign keys, cascade delete of alerts).
- **Maintainability:** access rules are expressed once, as reusable dependencies (`is_reportee`, `is_police`, `is_admin`), not repeated in each route.
- **Testability:** the database session is injected so tests can swap in an in-memory database.
- **Performance:** list endpoints are paged (default 20 reports, 30 users and alerts; maximum 100).

4.5 Use Cases / User Stories

UC-B1: Register and log in. A member of the public registers with name, email, telephone, national ID and password, and receives a token and a redirect to /my. On later visits they log in with email and password. *Alternate flows:* duplicate email (409); role other than reportee requested (400); password too short (400); wrong credentials (401); deactivated account (403).

UC-B2: File a report. A logged-in reportee submits plate and vehicle details. The server upper-cases the plate, refuses it if an active report for that plate exists (409), stores it as under_review, and writes an audit record.

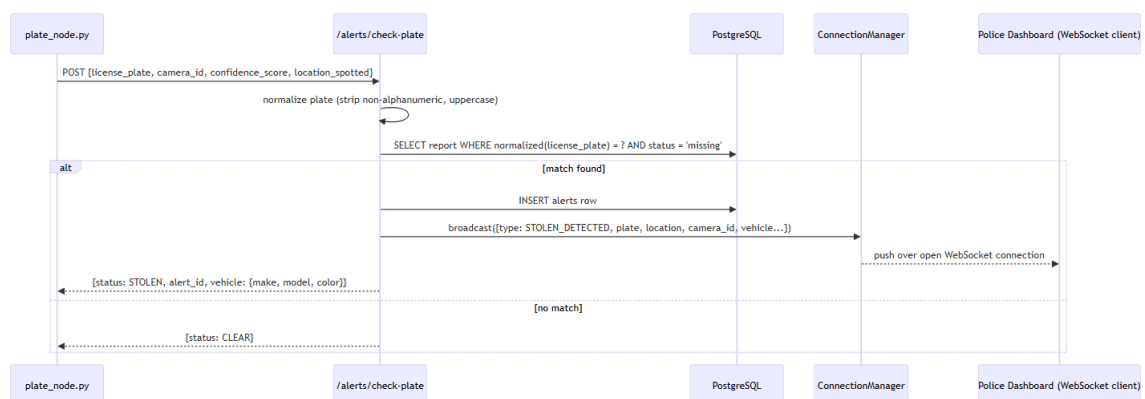
UC-B3: Police activate and resolve a report. An officer activates an under_review report so it becomes matchable, and later marks it found.

UC-B4: Camera checks a plate. The node sends a plate; the server returns CLEAR, or creates an alert and returns STOLEN with the vehicle's make, model and colour.

UC-B5: Administrator manages accounts. The administrator invites an officer, changes a role (a badge number is required for police or admin), or deactivates an account.

User story. *As a member of the public, I want to see only my own reports, so that other people's theft reports and personal details stay private.*

4.6 System / Solution Architecture

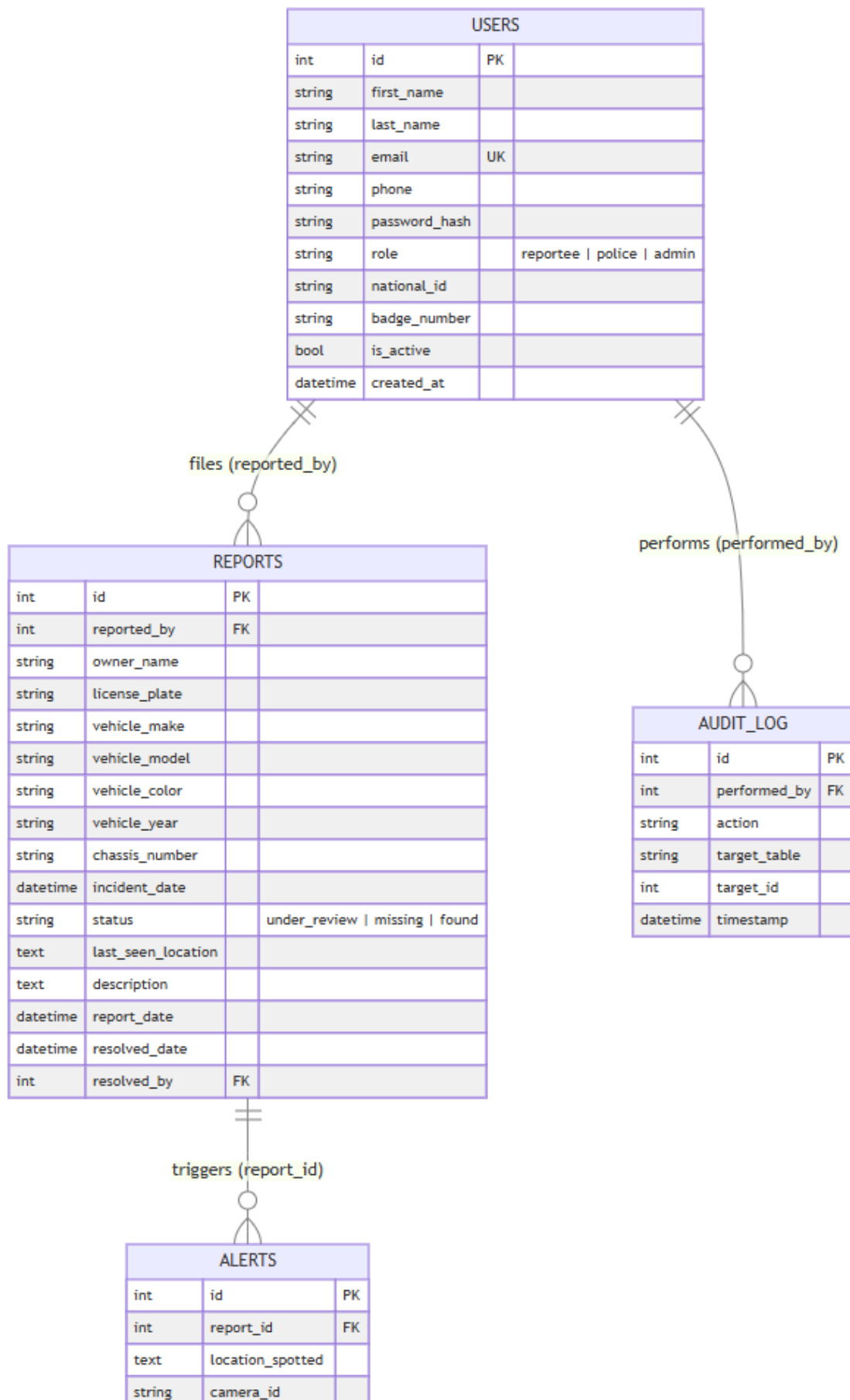


The backend is a single FastAPI application. main.py registers six routers (auth, reports, alerts, users, analytics, system) and the CORS middleware. Shared modules provide the database session (database.py), the ORM models (models.py), the validation schemas (schemas.py), password and token functions (auth.py), and the access-control dependencies (middleware.py).

```
vrrs-backend/app/  
  main.py          app, CORS, router registration, create_all()  
  database.py      engine, session, get_db  
  models.py        User, Report, Alert, AuditLog  
  schemas.py       Pydantic request/response models  
  auth.py          bcrypt + JWT  
  middleware.py    get_current_user, require_role, is_reportee/is_police/is_admin  
  routers/         auth, reports, alerts, users, analytics, system
```

4.7 Database / Data Design

Four tables, defined once in `models.py`.



float	confidence_score	
string	image_path	
bool	is_read	
datetime	detected_at	

Table	Purpose	Key constraints
users	Accounts	Primary key id; unique, indexed email; role string (reportee, police, admin); is_active flag; password_hash
reports	Stolen-vehicle reports	Foreign key reported_by → users; resolved_by → users; indexed license_plate; status string
alerts	Detections	Foreign key report_id → reports with ON DELETE CASCADE
audit_log	Action trail	Foreign key performed_by → users (nullable)

Design decisions: - **alerts.license_plate is not stored.** It is a computed property reading through the linked report, avoiding duplicated data. - **Cascade delete.** Deleting a report also deletes its alerts (the ORM relationship uses cascade="all, delete-orphan", and the database constraint uses ON DELETE CASCADE). - **Roles and statuses are plain strings**, not database enumerations or check constraints. This is simple, but the database itself does not stop an invalid value (§6.4). - **No migration tool.** Base.metadata.create_all() runs at start-up. It creates missing tables but never changes existing ones, so a constraint change on an existing table needs a manual ALTER TABLE.

4.8 Interface / Interaction Design

The backend's interface is its HTTP API, documented automatically at /docs. Access is by role (✓ allowed):

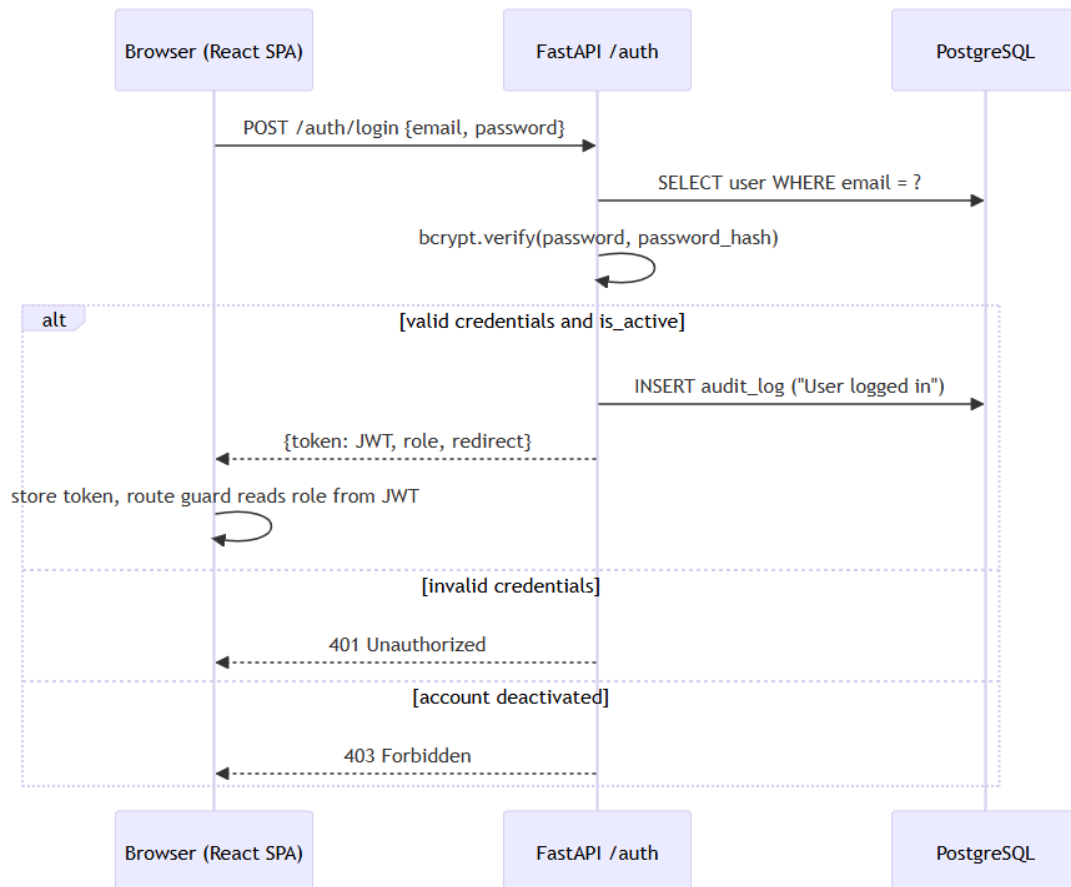
Route group	Endpoint(s)	Reportee	Police	Admin
Auth	POST /auth/register, /auth/login, /auth/logout	public	public	public
Reports	POST /reports/, GET /reports/, GET /reports/{id}	✓ (own only for reads)	✓ all	✓ all
Reports	PATCH /reports/{id}, /activate, /found		✓	✓
Reports	DELETE /reports/{id}			✓
Alerts	POST /alerts/check-plate	public	public	public

Route group	Endpoint(s)	Reportee	Police	Admin
Alerts	GET /alerts/, /{id}, /unread/count, PATCH .../read, /read-all, /false-positive		✓	✓
Users	GET/PATCH /users/me, PATCH /users/me/password	✓	✓	✓
Users	GET /users/, GET /users/{id}		✓	✓
Users	POST /users/invite, PATCH /users/{id}/role, /toggle-active, DELETE /users/{id}			✓
Analytics	GET /analytics/...		✓	✓ (user-growth admin only)
System	GET /system/audit			✓

The web pages that use these routes were designed by the frontend member.

4.9 Algorithms / Models / Technical Design

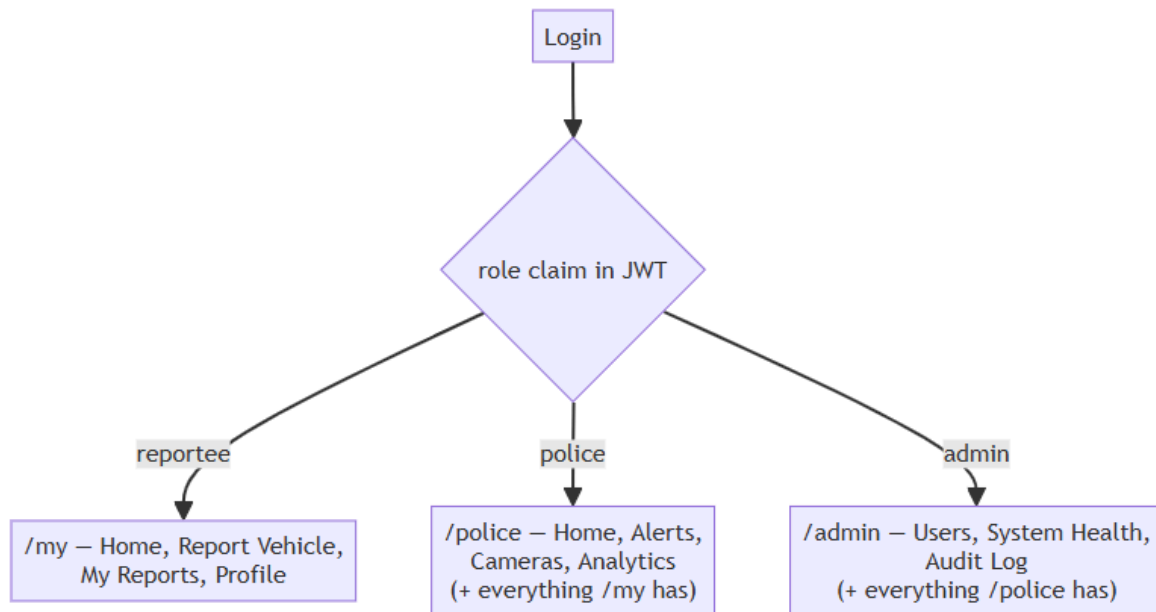
4.9.1 Authentication and token design



On login the server looks up the user by lower-cased email, verifies the password against the stored bcrypt hash, rejects deactivated accounts, writes an audit record, and returns a JWT whose claims are sub (user id), role and exp (default 24 hours, configurable). The token is signed with HS256 using SECRET_KEY from the environment. On each request, get_current_user decodes the token and returns the user id and role **taken from the token**.

4.9.2 Role enforcement

require_role(*roles) builds a dependency that returns 403 if the token's role is not in the list. Three shortcuts wrap it: is_reportee (all three roles), is_police (police and admin), is_admin (admin only). Routes attach them with Depends(...). Row-level rules are added inside the route: a reportee's list is filtered by reported_by, and reading another reportee's report returns 403.



4.9.3 Plate matching

check-plate removes every character except letters and digits from the received plate, upper-cases it, and compares it with the same normalisation applied to the stored plate in SQL (`regexp_replace(upper(license_plate), '^[A-Z0-9]', '', 'g')`). It matches only reports whose status is missing. This makes `abc 1234`, `ABC-1234` and `ABC1234` equal. On a match it inserts an alert, calls the WebSocket broadcast (the real-time member's component) and returns `STOLEN` with make, model and colour.

4.9.4 Report lifecycle

```
under_review --(police: activate)--> missing --(police: mark found)--> found
```

Only missing reports are matched. `activate` accepts only `under_review`; `found` refuses a report that is already found. The general `PATCH /reports/{id}` route is separate and is discussed in §6.4.

4.10 Chapter Summary

The backend is a FastAPI service over four PostgreSQL tables. Access control is expressed as three reusable role dependencies plus per-row ownership rules. Authentication uses bcrypt and a signed JWT that carries the role. The `plate-match` endpoint normalises both sides so formatting differences do not prevent a match.

Chapter 5: Implementation / Development

5.1 Introduction

This chapter describes how the backend was built: its modules, the key implementation decisions, how it connects to the other components, the problems met, and the student's own contribution.

5.2 Development Environment

Item	Detail
Machine	Windows 11 PC
Language and runtime	Python (the test virtual environment used Python 3.12.10)
Database	PostgreSQL (vrrs_db), connection string from DATABASE_URL
Frameworks	FastAPI 0.111.0, SQLAlchemy 2.0.30, Pydantic 2.7.1
Security libraries	passlib 1.7.4 + bcrypt 4.1.3, python-jose 3.3.0
Tools	Swagger UI at /docs for manual trials, pytest for automated tests, Git
Configuration	.env (not committed); .env.example lists DATABASE_URL, SECRET_KEY, ALGORITHM, TOKEN_EXPIRY_HOURS, CORS_ORIGINS

5.3 System Components / Modules

Module	Lines	Purpose	Built by
app/main.py	55	App creation, CORS, router registration, schema creation	Backend (this report)
app/database.py	20	Engine, session factory, get_db dependency	Backend (this report)
app/models.py	65	User, Report, Alert, AuditLog ORM models	Backend (this report)
app/schemas.py	132	Pydantic request and response models with validation patterns	Backend (this report)
app/auth.py	33	bcrypt hashing; JWT create and decode	Backend (this report)

Module	Lines	Purpose	Built by
app/middleware.py	34	get_current_user, require_role, role shortcuts	Backend (this report)
app/routers/auth.py	48	Register, login, logout	Backend (this report)
app/routers/users.py	125	Profile, password change, admin user management	Backend (this report)
app/routers/reports.py	94	Report create, list, read, update, activate, found, delete	Backend (this report)
app/routers/alerts.py	123	check-plate and alert listing (this report); WebSocket manager and endpoint (real-time member)	Shared
app/routers/analytics.py, system.py	87, 146	Statistics; audit-log listing, camera status, feed proxy, health	Frontend and real-time members
tests/	5 test files and conftest.py	Automated tests	See §5.7

Line counts are from the files as they stand.

5.4 Key Implementation Details

5.4.1 Password hashing and tokens (auth.py)

```

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password): return pwd_context.hash(password)
def verify_password(plain, hashed): return pwd_context.verify(plain, hashed)

def create_access_token(user_id, role):
    payload = {"sub": str(user_id), "role": role,
               "exp": datetime.utcnow() + timedelta(hours=TOKEN_EXPIRY_HOURS)}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

```

decode_token returns None on any signature or expiry error. The signing key is read from the environment. The code falls back to the string "changeme" if SECRET_KEY is unset; on the development machine the key is set and is not the default (checked, §6.4).

5.4.2 Access control (middleware.py)

```
def require_role(*roles):
    def checker(user: dict = Depends(get_current_user)):
        if user["role"] not in roles:
            raise HTTPException(status_code=403, detail=f"Access denied.
Required roles: {list(roles)}")
        return user
    return checker

is_reportee = require_role("reportee", "police", "admin")
is_police   = require_role("police", "admin")
is_admin    = require_role("admin")
```

(In the source these are small wrapper functions; they are shown here in condensed form.) A route needs one line to require a role, for example `user=Depends(is_admin)`.

5.4.3 Ownership and lifecycle rules (reports.py)

- `create_report` upper-cases and trims the plate, returns 409 if a missing report with that plate exists, saves the report as `under_review`, and writes an audit row.
- `get_reports` filters by `reported_by` when the caller is a reportee, and supports `status`, `search` (plate or owner name, case-insensitive), `skip` and `limit`.
- `get_report` returns 403 if a reportee asks for someone else's report.
- `activate_report` accepts only `under_review`; `mark_found` records `resolved_by` and `resolved_date`.
- `delete_report` is admin-only and relies on the cascade to remove alerts.

5.4.4 Plate matching (alerts.py)

```
normalized = re.sub(r"^[A-Z0-9]", "", data.license_plate.upper())
report = db.query(Report).filter(
    func.regexp_replace(func.upper(Report.license_plate), r"^[A-Z0-9]", "", "g")
    == normalized,
    Report.status == "missing",
).first()
if not report:
    return {"status": "CLEAR"}
new_alert = Alert(report_id=report.id, ...)
db.add(new_alert); db.commit()
await manager.broadcast({... "type": "STOLEN_DETECTED" ...})
return {"status": "STOLEN", "alert_id": new_alert.id, "vehicle": {...}}
```

5.4.5 Admin safeguards (users.py)

An administrator cannot change, deactivate or delete their own account (prevents locking out the last admin by accident). Assigning police or admin requires the target user to have a badge number. Password change requires the current password and rejects an unchanged or too-short new password. `UserUpdate`, the model used for profile edits, has no `role` or `is_active` field, so a user cannot promote themselves through their profile.

5.4.6 Audit records

Registration, login, report filing, updates, activation, found, deletion, account creation, role change, activation or deactivation of an account, password change, profile update and false-positive flags each add an `AuditLog` row with the actor's id, a text description, the target table and the target id.

5.5 Integration

- **Camera node (AI/vision member):** the only contract is `POST /alerts/check-plate` with `license_plate`, `camera_id`, `confidence_score`, `location_spotted` and optional `image_path`. The backend normalises the plate itself, so the node does not need to reproduce the matching rules.
- **Real-time member:** `check-plate` calls the `ConnectionManager.broadcast` method they wrote; the WebSocket handshake reuses `decode_token` and admits only police and admin tokens.
- **Frontend member:** the front end calls the routes in §4.8 with the token in an `Authorization: Bearer` header and reads the role from the login response. CORS is limited to the local development origins `localhost:5173` and `5174` (and `127.0.0.1` equivalents).

Note: `main.py` builds the CORS list from four hard-coded origins. `CORS_ORIGINS` appears in `.env.example` but is not read by the application code, so changing it has no effect (checked by searching the source).

5.6 Challenges and Solutions

Challenge	Cause	Solution
Deleting a report that had detections	The <code>alerts.report_id</code> foreign key was created with no delete action	Added <code>onDelete="CASCADE"</code> to the foreign key and <code>cascade="all, delete-orphan"</code> to the relationship. Because <code>create_all()</code> does not alter

Challenge	Cause	Solution
failed with a database error		existing tables, the live constraint was also altered by hand and the change verified in PostgreSQL's constraint catalogue. Covered by a regression test
A PostgreSQL-only function in <code>check-plate</code> could not run under the test database	SQLite has no four-argument <code>regexp_replace</code>	The test set-up registers a Python implementation on each SQLite connection, so the production query runs unchanged in tests
Cascade delete did not fire in tests	SQLite ignores foreign keys unless enabled	The test set-up runs <code>PRAGMA foreign_keys=ON</code> per connection
Tests must not touch the real database	<code>database.py</code> connects to PostgreSQL at import	The tests override the <code>get_db</code> dependency with an in-memory SQLite session using a static pool
Camera cannot log in	It has no user identity	<code>check-plate</code> is left public and limited in what it can do (it cannot create or alter reports); see §6.4 for the consequence
Roles enforced only in the UI would be bypassable	Clients can send any request	Every route declares its role dependency; a parametrised test hits police and admin routes as a reportee

5.7 Individual Contribution (group project)

The student was responsible for **work areas 3 and 4** of the group's seven (Appendix A): the backend API and database, and security and access control. This covers:

- the four-table data model and its relationships, keys and cascade;
- the FastAPI application structure and configuration;
- registration, login, password hashing and token issue and checking;
- the role dependencies and the ownership rules on reports;
- user management, including the admin safeguards;
- the report lifecycle endpoints and the plate-match endpoint;
- writing of audit records;
- the security review in Chapter 6.

Work by other members that this component uses, **not claimed here**: the WebSocket manager and endpoint, camera status, live-feed proxy and system health (real-time and integration member); analytics endpoints and the React pages (frontend member); the detection model and camera node (AI/vision member).

Tests. The group's role split placed "tests" with the frontend member. The 31 backend tests exercise this component, and the student ran them and interpreted their results for this report. **Who wrote each test file should be confirmed by the student before submission;** §6.3 describes them without claiming authorship.

5.8 Chapter Summary

The backend is about 730 lines of Python across ten modules (`main.py`, `database.py`, `models.py`, `schemas.py`, `auth.py`, `middleware.py` and four routers; the generic `model.py` placeholder is not counted). It hashes passwords with `bcrypt`, issues signed role-carrying tokens, enforces roles with reusable dependencies, and answers plate-match queries. Its notable implementation problems were the missing cascade and the gap between PostgreSQL and the SQLite test database.

Chapter 6: Testing, Results and Evaluation

6.1 Introduction

This chapter evaluates the backend. It reports the automated test results, then the security findings from targeted probes, then judges each individual objective. Every result below comes from output that was produced during the preparation of this report and can be re-run.

6.1.1 Version of the code evaluated

Note on the version of the code evaluated. All findings, test counts, coverage figures and line counts in this report were obtained against the code as committed (commit a349fc6). After that evaluation, uncommitted changes appeared in the working tree (middleware.py, alerts.py, system.py, main.py, api.js, SystemHealth.jsx, conftest.py, a new tests/test_regressions.py and a new front-end test). With those changes the backend suite has **38 tests, all passing**, and the front end has **one** test (npm test), which also passes. The changes appear to address the items listed below; this was established by reading the changes and running the tests, and the original probes were **not** re-run against the changed code. Where this report says a finding was "not fixed", it refers to the evaluated commit.

Appear addressed in the working tree: **F-1** and **F-2** (every request now re-reads the user from the database, so a deactivated or demoted user's old token is rejected, and the role comes from the database, not the token) and **F-9** (CORS_ORIGINS is now read). *Still open as far as the changes show:* F-3, F-4, F-5, F-6, F-7 (logout does not end a session by itself), F-8 and F-10.

6.2 Testing Strategy

Level	Technique	Target
Integration (API)	pytest with FastAPI TestClient, real routes, real ORM	Auth, RBAC, reports, plate matching, analytics
Security probing	Short temporary tests confirming suspected weaknesses	Token lifetime, status validation, public endpoint, login attempts, signing key, logout
Code review	Reading the source against OWASP categories	All backend modules
Acceptance / user testing	None	See §6.5

Test environment. The tests use an in-memory SQLite database, not PostgreSQL, and the real `get_db` dependency is replaced by an override. Two SQLite gaps are closed in `conftest.py`: foreign keys are switched on, and a Python version of PostgreSQL's four-argument `regexp_replace` is registered so the production matching query runs unchanged. This means results say nothing about behaviour that differs between SQLite and PostgreSQL (for example locking, data types or the constraint catalogue).

6.3 Test Cases and Results

The suite has **31 tests, all passing** (re-run while preparing this report; the original captured output is in `thesis/assets/test-results/backend-pytest-output.txt`).

Table 6.1: Automated test cases

ID	Area	Test case	Expected result	Status
TC-01	Auth	Register a reportee	201; role reportee; redirect /my	Pass
TC-02	Auth	Register with an existing email	Rejected	Pass
TC-03	Auth	Register asking for a non-reportee role	Rejected	Pass
TC-04	Auth	Register with a short password	Rejected	Pass
TC-05	Auth	Login with correct credentials	Token returned	Pass
TC-06	Auth	Login with a wrong password	401	Pass
TC-07	Auth	Login to a deactivated account	Rejected	Pass
TC-08	Auth	Protected route without a token	401	Pass
TC-09 to TC-12	RBAC	Reportee requests /users/, /system/audit, /alerts/, /analytics/summary	403 each	Pass (4 cases)
TC-13,	RBAC	Police request /system/audit, /analytics/user-growth	403 each	Pass (2 cases)

ID	Area	Test case	Expected result	Status
TC-14				
TC-15	RBAC	Police request /alerts/	200	Pass
TC-16	RBAC	Admin request /system/audit	200	Pass
TC-17	RBAC	Reportee tries to delete a report	403	Pass
TC-18	Reports	Reportee files a report	201; plate upper-cased; under_review	Pass
TC-19	Reports	A second report while the first is only under_review	201 (allowed; see §6.4 finding F-5)	Pass
TC-20	Reports	Filing against a plate with a missing report	409	Pass
TC-21	Reports	Reportee lists reports	Sees only their own	Pass
TC-22	Reports	Police activate then mark found	missing then found; resolved_by set	Pass
TC-23	Reports	Admin deletes a report that has an alert	204; report and alert both gone	Pass
TC-24	Plate match	Unknown plate	CLEAR	Pass
TC-25	Plate match	Exact plate on a missing report	STOLEN	Pass
TC-26	Plate match	Different case and spacing	STOLEN	Pass
TC-27	Plate match	Plate on a found report	CLEAR	Pass
TC-28	Alerts	Unread count and marking read	Counts update	Pass
TC-29	Alerts	Flag false positive	Audit row written	Pass

ID	Area	Test case	Expected result	Status
TC-30, TC-31	Analytics	Confidence averages across mixed scales	Normalised	Pass (owned by the analytics work; listed for completeness)

Counting the parametrised cases, this is 31 tests. TC-19 is worth reading closely: it is a test that *documents* current behaviour, and that behaviour is itself a weakness (F-5).

6.4 Security Findings from Probes

Seven probe tests were written as temporary tests, run against the in-memory database, and then deleted, so the shipped suite stays a specification of intended behaviour. They produced findings F-1 to F-4, F-6 and F-7 and the key check in F-8; one probe was invalid (see below) and is not reported. F-5 comes from an existing test (TC-19), not a probe.

Table 6.2: Findings

ID	Suspected weakness	What was observed	Severity (student's judgement)
F-1	A token stays valid after the user is deactivated	After setting <code>is_active = False</code> , the user's existing token still got 200 from GET <code>/users/me</code> , while a fresh login was refused with 403	High
F-2	A token keeps the role it was issued with	After an admin was demoted to reportee, their old token still got 200 from GET <code>/system/audit</code> (an admin-only route)	High
F-3	PATCH <code>/reports/{id}</code> accepts any status	A police user set <code>status</code> to "banana"; the server returned 200 and stored it. The same route can also set <code>missing</code> without going through <code>activate</code>	Medium
F-4	<code>check-plate</code> is unauthenticated and not rate-limited	Three identical unauthenticated calls for one active plate returned STOLEN three times and created three alert rows	Medium
F-5	The duplicate-report check ignores <code>under_review</code> reports	A second report for the same plate was accepted while the first was <code>under_review</code> (TC-19)	Low

ID	Suspected weakness	What was observed	Severity (student's judgement)
F-6	No limit on failed logins	Twenty wrong-password attempts against one account all returned 401; none was locked out or slowed	Medium
F-7	Logout does not invalidate the token	After POST /auth/logout, the same token still got 200 from GET /users/me	Medium

Three further observations came from reading and from checking configuration:

- **F-8 Signing key.** The code falls back to "changeme" if SECRET_KEY is unset. A check showed the development configuration uses a 47-character key that is not the default, and a token forged with "changeme" was rejected. The weakness is therefore *latent* (a deployment that forgets to set the variable would be forgeable), not active.
- **F-10 Admin seed script.** vrrs-backend/create_admin.py, the script that creates the first administrator, falls back to a hard-coded default password if ADMIN_PASSWORD is not set, prints the credentials to the console, and is committed to the repository. Anyone who runs it without setting the variable creates an administrator with a password that is public in the source. It also defaults DATABASE_URL to a local SQLite file if none is set. This was found by reading the script; it was not run.
- **F-9 CORS configuration.** CORS_ORIGINS in .env.example is not used; origins are hard-coded to localhost. This is safe for development but means the setting does nothing, and a real deployment would need a code change.

A probe that was invalid. A probe intended to test whether a six-character password such as abcdef is accepted at registration used an incomplete payload (missing the required national ID) and returned 422 for that reason. It shows nothing about the password rule, so **no finding is claimed**. The password minimum (six characters, enforced in the route) is covered by TC-04. Six characters is short by current guidance and there is no complexity check, which is a design observation, not a probe result.

Additional points from reading the code (not probed):

- Roles and statuses are free-text columns with no database check constraint, and the role and status are not validated by an enumeration in the schemas (ReportUpdate.status is Optional[str]), which is the root of F-3.
- email is a plain string, not validated as an email address.
- UserUpdate has no format pattern for phone or national ID, unlike registration, so profile edits can store malformed values.

- Failed logins are not written to the audit log; successful ones are.
- Many list and detail routes are async functions that call blocking database code; under load this could stall the event loop. This was not measured.
- In `delete_report`, the audit row is written *after* the delete is committed, so a failure between the two would leave a deletion with no record.

6.4.1 What was not tested

No load or performance test; no PostgreSQL-specific test; no penetration test; no test of the WebSocket or live-feed proxy (other member); no test of TLS or deployment configuration; no automated test of the front end.

6.5 User Evaluation

None carried out. No officers, administrators or members of the public used the backend directly, and no survey or interview data exist.

6.6 Analysis of Results

- **The intended behaviour is well covered.** The role matrix, ownership rule, lifecycle, cascade delete and plate normalisation all have passing tests, so the core promises of Chapter 4 hold under test.
- **The weaknesses cluster around the token design.** F-1, F-2 and F-7 have one cause: the server trusts the token's contents (`role`, and the fact that it was issued) and does not re-check the database on each request. That is the stateless trade-off named in §2.2.4. In practice it means a deactivated or demoted police officer keeps their old access until the token expires (24 hours by default). For a police system that is the most important finding.
- **F-3 and F-5 are validation gaps**, not access-control failures: the people who can do these things are already trusted (police), but the data can end up in states the rest of the system does not expect.
- **F-4 is a consequence of a design choice** (a public endpoint for a camera with no identity). The endpoint cannot alter reports, but anyone who can reach it can flood the alert table and the police dashboard with false alerts for any active plate they know.
- **F-6** matters more if passwords are short, as here.
- **The tests cannot see PostgreSQL-specific problems**, so passing tests are evidence about logic, not about the deployed database.

6.7 Objective-by-Objective Evaluation

Objective	Evidence	Verdict
B1 Data model	Four tables with keys, unique email, cascade; tests create the schema and the cascade test passes. <code>create_all()</code> only, no migrations	Achieved , with the migration limitation
B2 Registration, login, hashing, JWT	bcrypt hashes; token with <code>sub</code> , <code>role</code> , <code>exp</code> ; TC-01 to TC-08 pass	Achieved ; token revocation and lockout not provided (F-1, F-6, F-7)
B3 Server-side role enforcement	Role dependencies on every route; role matrix TC-09 to TC-17 pass	Achieved for the token as issued; weakened by F-1 and F-2, which let a stale role persist
B4 Report lifecycle and ownership	TC-18 to TC-23 pass	Largely achieved ; status validation missing (F-3) and duplicate check incomplete (F-5)
B5 Plate-match endpoint	TC-24 to TC-27 pass	Achieved ; public and unthrottled (F-4)
B6 Audit records	Rows written for the listed actions; TC-29 checks one	Largely achieved ; failed logins not recorded; only one action is directly tested
B7 Verification and security review	31 passing tests; ten findings recorded	Achieved for this scope; no penetration test

6.8 Individual Results

All results in this chapter concern the backend and security component, except TC-30 and TC-31 (analytics), which are shown for completeness and are not claimed. The camera-node tests, the WebSocket and the front end are in the other members' reports.

6.9 Chapter Summary

All 31 tests pass. Six weaknesses were confirmed by probes and one (F-5) by an existing test, the most serious being that tokens outlive deactivation and demotion. Ten findings in total, including three from configuration and code reading, were recorded and none has been fixed in this project; they are listed as recommendations in Chapter 8.

Chapter 7: Discussion

7.1 Introduction

This chapter interprets the results of Chapter 6, compares them with related practice, states the contributions, and discusses limitations.

7.2 Interpretation of Findings

The access-control design works as specified, for the token as issued. Every protected route rejects a missing token, and the role matrix behaves as designed. Reportees cannot read each other's reports, only administrators can delete, and administrators cannot lock themselves out. This answers research question 1 in the affirmative: role-based access can be enforced consistently on the server, using a signed token that carries the role, without relying on the front end.

The same design is what limits it. Research question 2 asked what weaknesses remain. The main one is that the token is trusted for its whole lifetime. A deactivated officer or a demoted administrator keeps working until the token expires, and logging out does not end a session. These are not coding mistakes in the role checks; they follow directly from choosing stateless tokens without a revocation check. The fix is small (look up the user in the database on each request, or keep a token deny-list), but it costs one query per request, which is the trade the design originally avoided.

Validation is thinner than authorisation. The role checks are consistent, but free-text status and role columns, a plain-string email field and an unvalidated status update let bad values in from people who are already trusted. This is typical of a first version: access control was designed carefully because it is the headline requirement, and data validation was left to the front end.

The public camera endpoint is the sharpest design trade-off. Making check-plate public keeps the camera node simple, and it cannot create or edit reports. But it lets anyone who can reach the server, and who knows or guesses an active plate, generate real-looking alerts on police screens. The risk is bounded by network placement, which the code does not enforce.

7.3 Comparison with Existing Work

No comparison of measured results with other systems was made, because none of the systems reviewed in Chapter 2 publish comparable security test results for a stolen-vehicle registry. On design, this backend implements a small version of what hosted identity providers and mature frameworks provide (hashing, signed tokens, roles) and lacks what they add (revocation, refresh tokens, multi-factor authentication, rate limiting). That is the expected difference between a student prototype and a hardened service. The OWASP categories used as a checklist line up with the findings: broken access control (F-1, F-2), identification and authentication failures (F-6, F-7), security misconfiguration (F-8, F-9) and insufficient input validation (F-3).

7.4 Contributions of the Project

- **Practical:** a working, documented API with role-scoped access, a report lifecycle, an audit trail and a plate-match endpoint that tolerates formatting differences.
- **Technical:** an access-control layer expressed once as reusable dependencies; a test set-up that runs the production PostgreSQL query on SQLite and checks the cascade.
- **Analytical:** a verified list of what this style of design does *not* protect against, with a reproducible probe for each.

7.5 Limitations and Implications

Limitation	Implication
Tests run on SQLite	Passing tests do not show PostgreSQL behaviour; the constraint and data-type behaviour of the real database was checked only for the cascade fix
Probes were written by the author of the code	They test the weaknesses the author thought of. Other weaknesses may exist
No penetration test or external review	The findings are a lower bound
No load testing	Behaviour under many simultaneous requests, including the <code>async-with-blocking-code</code> concern, is unknown
Severity ratings are the student's own judgement	They are not a formal scoring such as CVSS
Findings not fixed	The delivered system still contains them (§8.5)
No stakeholder input	Requirements may miss needs of real officers

7.6 Chapter Summary

The backend meets its access-control goals for tokens as issued, and the tests show it. Its weaknesses are concentrated in token lifetime, input validation and the public endpoint, and each was demonstrated or read from the code. They are documented here so they can be fixed, not hidden.

Chapter 8: Conclusion and Recommendations

8.1 Introduction

This chapter summarises the project, states the conclusions the evidence supports, reviews the objectives, and gives recommendations and future work.

8.2 Summary of the Project

The group built SVDS so that stolen-vehicle reports filed by the public can be matched automatically against plates seen by a camera. This report covered the backend and security component: a FastAPI service over a four-table PostgreSQL model, with bcrypt password hashing, signed role-carrying tokens, server-side role checks, a report lifecycle, a plate-match endpoint for the camera node, and an audit trail. It was verified by 31 automated tests and by targeted security probes.

8.3 Conclusions

1. Server-side role-based access control with a signed token is achievable with a small amount of code and behaves correctly under test.
2. Passwords are stored as bcrypt hashes and the reportee, police and admin roles are separated as designed.
3. The plate-match endpoint works across case and spacing differences and matches only active reports.
4. The design has confirmed weaknesses: tokens outlive deactivation, demotion and logout; report status is not validated; the public camera endpoint can be used to create false alerts; and there is no login throttling.
5. The system is a good prototype and is **not** ready for production use with real police data until the recommendations below are addressed.

8.4 Achievement of Objectives

Objective	Status
B1 Data model	Achieved (no migrations)
B2 Authentication and password storage	Achieved; revocation and lockout missing

Objective	Status
B3 Server-side role enforcement	Achieved for tokens as issued; weakened by stale-role tokens
B4 Report lifecycle and ownership	Largely achieved; status validation and duplicate check incomplete
B5 Plate-match endpoint	Achieved; public and unthrottled
B6 Audit records	Largely achieved; failed logins not recorded
B7 Verification and security review	Achieved for this scope

Research question 1: yes. Research question 2: ten weaknesses were found and recorded.

8.5 Recommendations

In priority order:

1. **Re-check the user on every request** (or keep a token deny-list) so deactivation, demotion and logout take effect immediately; shorten the token lifetime and add refresh tokens (F-1, F-2, F-7).
2. **Validate report status** with an enumeration in the schema and a database check constraint, and route status changes through the lifecycle endpoints only (F-3). Do the same for role.
3. **Protect check-plate** with an API key or a private network, and add rate limiting and de-duplication of repeated alerts (F-4).
4. **Throttle failed logins**, log them in the audit trail, and raise the password minimum with a basic strength check (F-6).
5. **Fail at start-up if SECRET_KEY is unset** instead of using a default (F-8), and read allowed origins from configuration (F-9).
6. Remove the default password from `create_admin.py`, require `ADMIN_PASSWORD`, and stop printing it (F-10).
7. Extend the duplicate check to `under_review` reports (F-5).
8. Adopt Alembic migrations and run the test suite against PostgreSQL.
9. Validate email addresses, and apply the registration formats to profile edits.
10. Write the audit row in the same transaction as the action it records.

8.6 Future Work

- Fix the findings above and turn each probe into a permanent regression test.
- Add multi-factor authentication for police and admin accounts.

- Add a structured penetration test and a load test.
- Add pagination totals and stronger search to the report lists.
- Move blocking database calls out of async routes, or use an async database driver.
- Define data retention and access-request rules that meet Zambian data protection requirements.

References

1. FastAPI. "FastAPI Documentation." <https://fastapi.tiangolo.com> (accessed 2026).
2. SQLAlchemy. "SQLAlchemy Documentation." <https://www.sqlalchemy.org>
3. PostgreSQL Global Development Group. "PostgreSQL Documentation." <https://www.postgresql.org/docs/>
4. Pydantic. "Pydantic Documentation." <https://docs.pydantic.dev>
5. Jones, M., Bradley, J., Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
6. Provos, N., Mazieres, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference.
7. Passlib. "Passlib Documentation." <https://passlib.readthedocs.io>
8. OWASP Foundation. "OWASP Top 10" and "OWASP API Security Top 10." <https://owasp.org>
9. Ferraiolo, D., Kuhn, D. R. (1992). Role-Based Access Controls. 15th National Computer Security Conference.
10. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.
11. pytest. "pytest Documentation." <https://docs.pytest.org>
12. Sommerville, I. (2011). Software Engineering, 9th ed. Pearson.

[Add any interviews, Zambia Police Service correspondence, the Zambian data protection legislation you rely on, or supervisor notes, and format all entries to the department's required style.]

Appendix A: Individual Contribution Statement (Group Projects)

Purpose of this Statement

This statement identifies the work for which the student was primarily responsible. It is consistent with the main report. Work by other group members is acknowledged and is not claimed.

A.1 Project and Group Information

Item	Details
Project Title	SVDS: A Camera-Based Stolen Vehicle Detection System for the Zambia Police Service (built on the VRRS registry)
Student Name	[STUDENT NAME]
Computer Number	[COMPUTER NUMBER]
Supervisor	[SUPERVISOR NAME]
Group Size	4
Group Members	1. Moomba Nicholas Katapazi, [COMPUTER NUMBER] (AI/vision) 2. [STUDENT NAME], [COMPUTER NUMBER] (Backend and security) 3. [NAME], [COMPUTER NUMBER] (Real-time and integration) 4. [NAME], [COMPUTER NUMBER] (Frontend, analytics, testing and documentation)

A.2 Overall Group Project

The group built **SVDS/VRRS**, a system with which the public files stolen-vehicle reports and police officers are alerted automatically when a camera sees a reported vehicle. It has three independently running processes: a FastAPI and PostgreSQL backend, a React frontend with three role-based portals (Reportee, Police, Admin), and a camera node that reads number plates from a phone's video stream using a custom-trained YOLOv8 detector and OCR. The common problem is that stolen-vehicle recovery depends on officers chancing to recognise a plate. The system's core flow is: camera node reads a plate, the backend matches it against active reports, and connected police dashboards receive an alert over WebSocket.

A.3 Individual Role and Responsibilities

The group's work was divided into four roles. The student's role was **Role 2: Backend and security**, which covers work areas 3 (backend API and database) and 4 (security and access control).

No.	Area of Responsibility	Description
3	Backend API and database	FastAPI service and PostgreSQL schema; report CRUD and lifecycle; the <code>/alerts/check-plate</code> matching endpoint
4	Security and access control	JWT login, bcrypt password hashing, Reportee/Police/Admin roles and middleware, user management, audit records

Other roles, for reference: **Role 1** the plate-detection model and camera node; **Role 3** WebSocket alerts, camera status and live-feed proxy, system health, and the phone-to-server wiring; **Role 4** the React portals, analytics, integration tests, run scripts and documentation.

A.4 Detailed Individual Contribution

- **Data design:** designed the four-table schema (users, reports, alerts, audit log), keys, unique email and the cascade from reports to alerts.
- **Software development:** wrote the FastAPI application structure, database session handling, ORM models and Pydantic schemas with telephone and national ID formats.
- **Security:** implemented bcrypt password storage, JWT issue and checking, and the `require_role` dependencies; implemented ownership rules on reports and the admin safeguards on user management.
- **Business logic:** implemented the report lifecycle (under_review to missing to found), the duplicate-active-report rule, and the plate-match endpoint with case and spacing normalisation.
- **Auditing:** added audit records to state-changing actions.
- **Defect fixing:** fixed the report-delete cascade defect.
- **Testing and security review:** ran the 31-test suite, wrote and ran the security probes, and recorded ten findings with recommendations.
- **Documentation:** this report, including the API and role tables.

A.5 Individual Deliverables

No.	Deliverable	Student's Contribution
1	Database schema and ORM models (<code>models.py</code> , <code>database.py</code>)	Designed and implemented
2	Authentication and access control (<code>auth.py</code> , <code>middleware.py</code> , <code>routers/auth.py</code> , <code>routers/users.py</code>)	Designed and implemented
3	Report lifecycle and plate-match endpoint (<code>routers/reports.py</code> , <code>check-plate</code> in <code>routers/alerts.py</code>)	Designed and implemented
4	Request and response schemas (<code>schemas.py</code>)	Designed and implemented
5	Security review: probe results and findings F-1 to F-10	Carried out and documented

A.6 Contribution to the Main System or Research Output

The student produced the server that stores and protects the system's data and answers the camera's question "is this plate stolen?". The camera node itself was built by the AI/vision member; the WebSocket delivery, camera status and live-feed proxy by the real-time and integration member; the web pages and analytics by the frontend member. The check-plate request and response format was agreed with the AI/vision member. The WebSocket manager lives in the same file as check-plate (`routers/alerts.py`) and was written by the real-time member.

A.7 Individual Implementation and Technical Work

Modules implemented by the student: `main.py`, `database.py`, `models.py`, `schemas.py`, `auth.py`, `middleware.py`, `routers/auth.py`, `routers/users.py`, `routers/reports.py`, and the check-plate and alert-listing routes in `routers/alerts.py`. Databases and integrations: PostgreSQL schema, the camera-node contract, and the hook that calls the real-time broadcast.

A.8 Individual Testing and Evaluation Contribution

The student ran the 31-test backend suite, wrote and ran the security probe tests, interpreted the results, and wrote Chapters 6 and 7. **Authorship of each file in `vrrs-backend/tests/` should be confirmed by the student:** the group's role split placed "tests" with the frontend member, so this report does not claim the suite as the student's own work. The camera-node tests and the manual browser checks belong to other members.

A.9 Collaboration with Other Group Members

- The **AI/vision member's** camera node calls POST /alerts/check-plate; its request and response format was agreed jointly.
- The **real-time and integration member's** WebSocket manager is called by check-plate and reuses decode_token; the WebSocket admits only police and admin tokens.
- The **frontend member's** pages call every route in Chapter 4 and depend on the role and redirect returned at login.

A.10 Contribution Summary

1. **Work primarily completed by the student:** the data model, authentication and access control, the report lifecycle, the plate-match endpoint, audit records and the security review.
2. **Work completed jointly with other members:** the camera-node contract; the alert broadcast hook; start-up through run-all.ps1.
3. **Work primarily completed by other members and used here:** the detection model and camera node; the WebSocket delivery, camera status and health endpoints; the web portals and analytics.

A.11 Estimated Contribution

The percentages below are the student's own estimate and must be completed by the student. They were not derived from any record.

Project Activity	Individual Contribution	Collaborative Contribution
Problem Analysis	[%]	[%]
Literature Review	[%]	[%]
Requirements Analysis	[%]	[%]
System/Research Design	[%]	[%]
Implementation / Development	[%]	[%]
Testing and Evaluation	[%]	[%]
Documentation	[%]	[%]
Other	[%]	[%]

A.12 Statement of Authorship and Contribution

I confirm that the contribution described in this statement accurately represents the work that I personally undertook as part of the group project. I have identified collaborative work and have not knowingly claimed the work of another group member as my own.

Student Name: [STUDENT NAME] **Computer Number:** ____ **Signature:** ____ **Date:** ____

A.13 Supervisor Verification

I confirm that, to the best of my knowledge, the contribution described above is consistent with the student's participation in the group project.

Supervisor Name: ____ **Signature:** ____ **Date:** ____

Appendix B: User Manual / Installation Guide

This guide covers the backend. The camera node and front end are documented in the other members' reports and the project README.md.

B.1 Requirements

- Python 3.12 or later, and PostgreSQL with an empty database (for example vrrs_db).
- The packages in vrrs-backend/requirements.txt.

B.2 Install and configure

```
cd vrrs-backend
pip install -r requirements.txt
cp .env.example .env
```

Edit .env:

Key	Meaning	Note
DATABASE_URL	PostgreSQL connection string	e.g. postgresql://postgres:<password>@localhost:5432/vrrs_db
SECRET_KEY	Token signing key	Must be a long random value. The code falls back to changeme if it is missing (finding F-8)
ALGORITHM	JWT algorithm	HS256
TOKEN_EXPIRY_HOURS	Token lifetime	24 by default
CORS_ORIGINS	Listed in the template but not read by the code ; allowed origins are hard-coded in main.py (finding F-9)	

B.3 Run

```
uvicorn app.main:app --reload
```

The API runs at http://localhost:8000. Interactive documentation is at /docs. Tables are created automatically at start-up if they do not exist.

Public registration creates reportees only, so the first administrator is created with the helper script `create_admin.py`:

```
set ADMIN_EMAIL=admin@example.local
set ADMIN_PASSWORD=<choose a strong password>
python create_admin.py
```

Always set `ADMIN_PASSWORD` yourself. If it is unset the script uses a default password that is written in its source (finding F-10). The script also defaults to a local SQLite file when `DATABASE_URL` is not set, so set that too. Later role changes are made by an administrator, and a promoted account must have a badge number.

B.4 Roles at a glance

Role	Can do
Reportee	Register, log in, file reports, see own reports, edit own profile
Police	Everything a reportee can, plus see all reports and users, activate and mark reports found, edit reports, view and flag alerts, view analytics
Admin	Everything police can, plus create police accounts, change roles, deactivate or delete users, delete reports, read the audit log

B.5 Troubleshooting

Symptom	Likely cause	Action
401 Invalid or expired token	Missing, expired or wrongly signed token	Log in again; check <code>SECRET_KEY</code> has not changed since the token was issued
403 Access denied	Role too low for the route	Use an account with the required role
409 when filing a report	A missing report for that plate exists	Check the existing report
Start-up prints "error creating DB schema"	Cannot reach PostgreSQL	Check <code>DATABASE_URL</code> and that PostgreSQL is running
Frontend blocked by CORS	Its origin is not in the hard-coded list	Add the origin in <code>main.py</code>
Deleting a report fails on an older database	The cascade constraint was created before the fix	Alter the <code>alerts.report_id</code> foreign key to <code>ON DELETE CASCADE</code> by hand

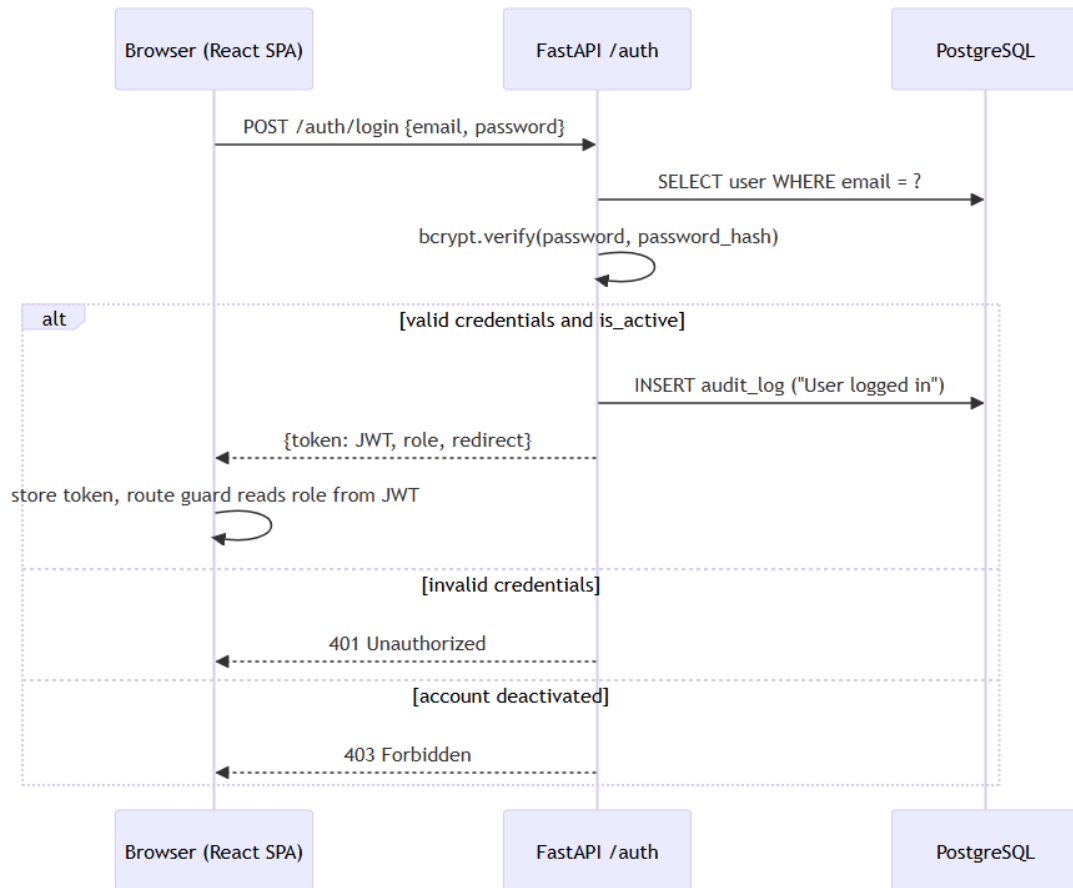
B.6 Run the tests

```
cd vrrs-backend  
python -m pytest tests/ -v
```

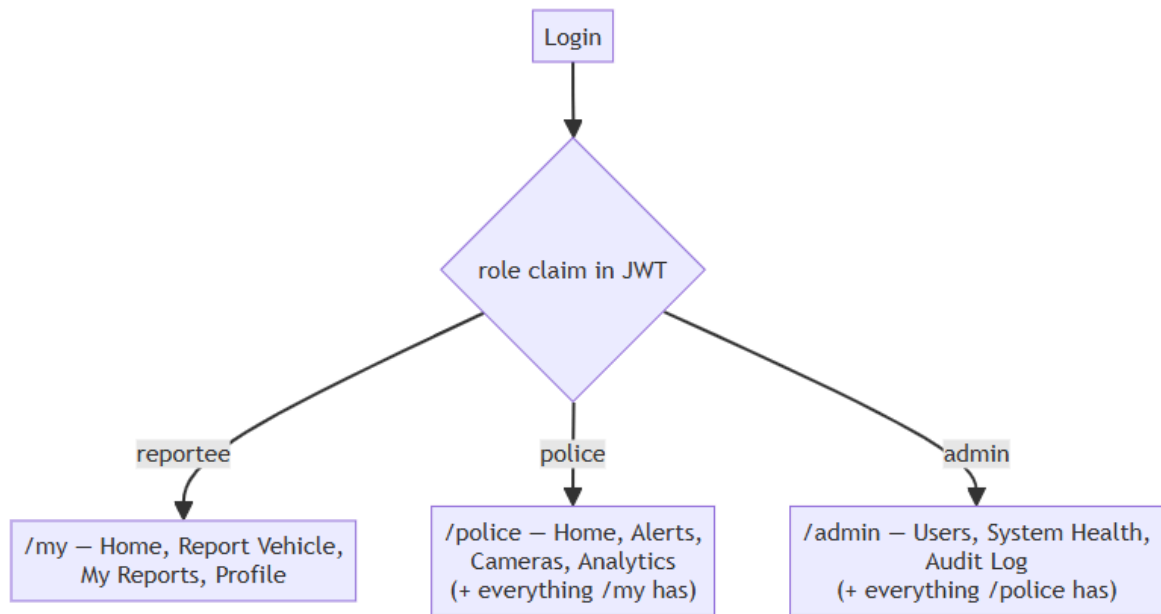
The tests use an in-memory SQLite database and do not touch PostgreSQL.

Appendix C: Additional System Designs / Diagrams

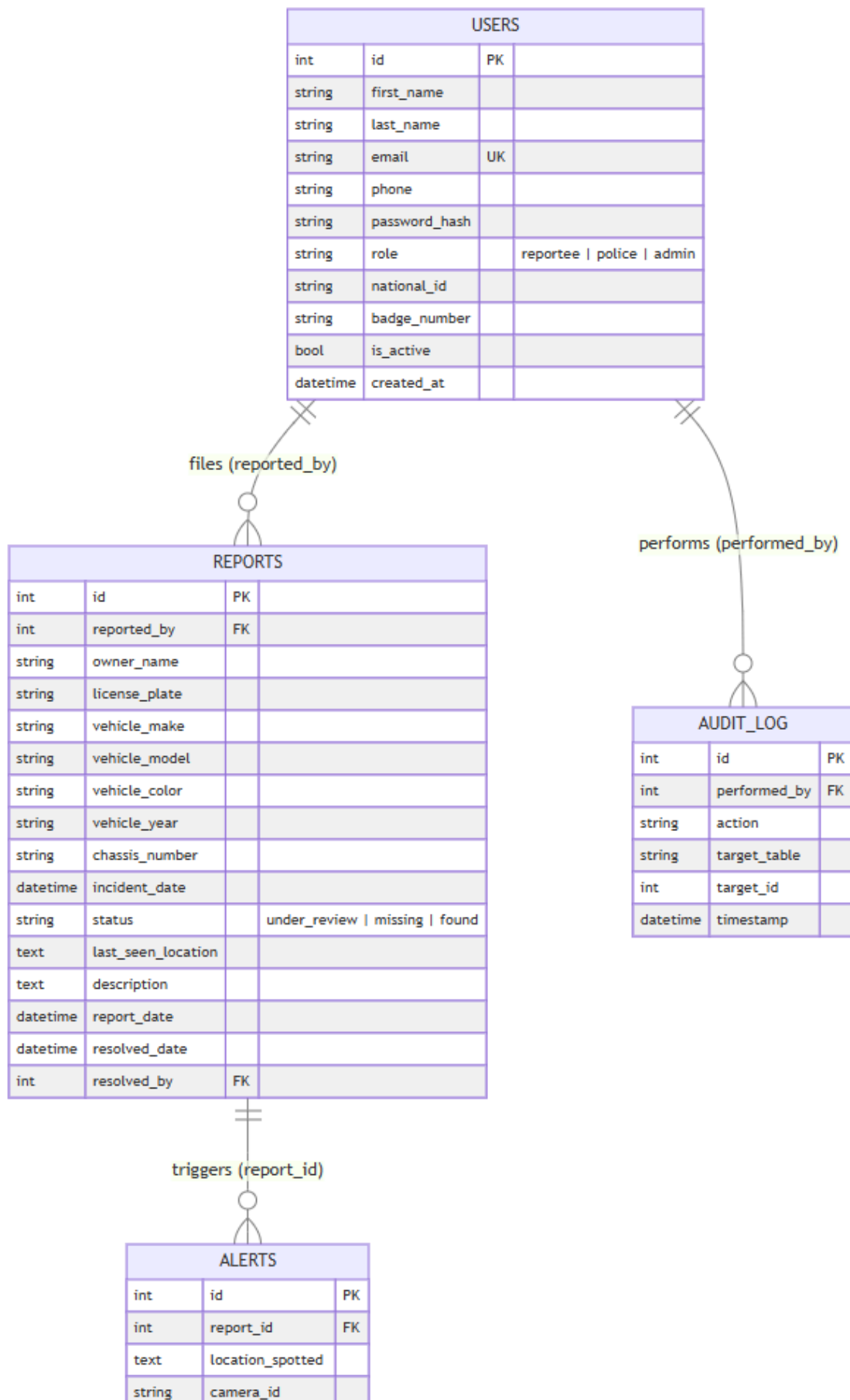
C.1 Authentication sequence



C.2 Role-based route access

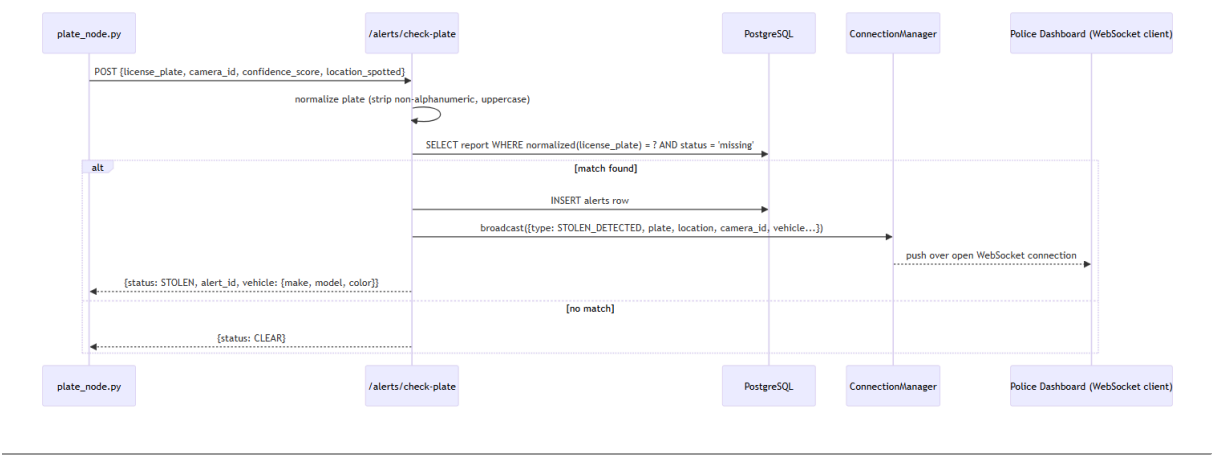


C.3 Database schema



float	confidence_score	
string	image_path	
bool	is_read	
datetime	detected_at	

C.4 Detection-to-alert sequence



Appendix D: Additional Test Cases and Results

D.1 Raw test output

The full captured output of the 31 passing tests is in `thesis/assets/test-results/backend-pytest-output.txt`. The suite was re-run while preparing this report and again passed (31 passed).

D.2 Probe results (raw)

The probe tests were temporary and were deleted after running; their printed output is reproduced below.

```
PROBE deactivated-user token /users/me          -> 200
PROBE demoted-admin old token /system/audit     -> 200
PROBE PATCH status=banana                      -> 200 banana
PROBE check-plate no-auth x3                   ->
['STOLEN','STOLEN','STOLEN'] alerts rows: 3
PROBE 20 wrong logins status set                -> {401}
PROBE register password 'abcdef'               -> 422 (invalid probe:
payload incomplete)
PROBE default-key forged token accepted         -> False
PROBE SECRET_KEY equals 'changeme'             -> False | length 47
PROBE token after /auth/logout                 -> 200
```

D.3 One probe as an example

```
def test_probe_deactivated_token_still_works(client, make_user, auth_headers,
db_session):
    u = make_user(role="reportee"); h = auth_headers(u)
    u.is_active = False; db_session.commit()
    print(client.get("/users/me", headers=h).status_code)          # observed:
200
    assert client.post("/auth/login",
        json={"email": u.email, "password": "Password123"}).status_code == 403
```

Appendix E: Data Collection Instruments

Not applicable. No questionnaires, interviews or surveys were used.

Appendix F: Additional Code / Configuration

F.1 Role dependencies (app/middleware.py)

```
def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = decode_token(token)
    if not payload:
        raise HTTPException(status_code=401, detail="Invalid or expired token.",
                             headers={"WWW-Authenticate": "Bearer"})
    return {"id": int(payload["sub"]), "role": payload["role"]}

def require_role(*roles):
    def checker(user: dict = Depends(get_current_user)):
        if user["role"] not in roles:
            raise HTTPException(status_code=403,
                                detail=f"Access denied. Required roles: {list(roles)}")
        return user
    return checker
```

F.2 Where finding F-1 and F-2 come from

`get_current_user` returns the role from the token and never reads the database, so a changed role or a deactivated account is not seen until the token expires. A fix would look up user by id here and reject inactive users.

F.3 Environment template (.env.example)

```
DATABASE_URL=postgresql://postgres:yourpassword@localhost:5432/vrrs_db
SECRET_KEY=your-super-secret-key-change-this-in-production
ALGORITHM=HS256
TOKEN_EXPIRY_HOURS=24
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Appendix G: Other Supporting Material

G.1 Group role split

Role	Work areas	Focus
1. AI/vision	Detection model; camera node	YOLOv8n detector, OCR pipeline
2. Backend and security	Backend API and database; security and access control	This report
3. Real-time and integration	Real-time alerts and monitoring; phone-to-server wiring	WebSocket, camera status, feed proxy, health
4. Frontend	Frontend portals; analytics, testing and documentation	React pages, analytics, run scripts

G.2 Findings summary

ID	Finding	Severity
F-1	Token valid after deactivation	High
F-2	Token keeps old role after demotion	High
F-3	Report status accepts any value	Medium
F-4	Public check-plate creates repeat alerts	Medium
F-5	Duplicate check ignores under_review	Low
F-6	No limit on failed logins	Medium
F-7	Logout does not invalidate token	Medium
F-8	Default signing key fallback (latent)	Latent
F-9	CORS_ORIGINS setting unused	Low
F-10	create_admin.py default password in source	Medium